CNNs and Computer Vision with PyTorch: Architecture, Transfer Learning, and Data Augmentation

Deep Learning Cohort I 2026 · Homework 3–4 Report (Weeks 3–4)

Bayram Bayramov
Student ID: 121314 Group: M003
bayramovb578@gmail.com

Abstract
This report presents an experimental study of convolutional neural networks for image classification on a CIFAR-10 subset with 5 classes. I implemented a SmallCNN from scratch (70.30% test accuracy), compared ResNet-18 feature extraction (86.40%) versus fine-tuning (93.70%), and evaluated data augmentation strategies including Mixup. The best performance was achieved by fine-tuning ResNet-18 with no augmentation (94.00%), demonstrating that pretrained features can be highly effective on small datasets. The key lesson is that transfer learning with fine-tuning dramatically outperforms training from scratch on limited data, and augmentation strategies should be chosen carefully as they can sometimes hurt performance.

Index Terms—convolutional neural networks, transfer learning, data augmentation, image classification, PyTorch.

I. Introduction

Convolutional neural networks are well-suited for vision tasks due to two key properties: parameter sharing (the same filter is applied across all spatial locations) and local connectivity (each neuron connects only to a local region of the input). This report covers a complete experimental study of CNNs on a CIFAR-10 subset, including a small CNN trained from scratch, transfer learning with ResNet-18, and a data-augmentation study.

This work maps directly to the homework: Section II presents a theoretical analysis (Part A), Sections III and IV describe and evaluate the experiments (Part B), and Section V explains the implementation details (Part C).

Contributions:

- a `SmallCNN` baseline trained on a CIFAR-10 subset;
- a comparison of ResNet-18 *feature extraction* vs. *fine-tuning*;
- an augmentation study including self-implemented Mixup.

II. Background and Theoretical Analysis

Throughout we use the convolution output-size formula

$$n_{\text{out}} = \left\lfloor \frac{n + 2P - k}{s} \right\rfloor + 1. \tag{1}$$

A. Convolution output size and parameter count

An RGB input of shape $3 \times 64 \times 64$ passes through a convolution with $C_{\text{out}} = 16$, $k = 5$, $s = 1$, $P = 2$, then a 2×2 max-pool ($s = 2$).

(a) Convolution:

$$n_{\text{out}} = \left\lfloor \frac{64 + 2(2) - 5}{1} \right\rfloor + 1 = \lfloor 63 \rfloor + 1 = 64 \tag{2}$$

Shape after conv: $16 \times 64 \times 64$.

Max-pool: $n_{\text{out}} = \lfloor (64 - 2)/2 \rfloor + 1 = 31 + 1 = 32$. Shape after pool: $16 \times 32 \times 32$.

(b) Learnable parameters:

$$(k^2 \cdot C_{\text{in}} + 1) \cdot C_{\text{out}} = (5^2 \cdot 3 + 1) \cdot 16 = (75 + 1) \cdot 16 = 1{,}216 \tag{3}$$

(c) Dense layer: flatten $3 \times 64 \times 64 = 12{,}288$ to 16 outputs: $12{,}288 \times 16 + 16 = 196{,}624$ parameters. Ratio: $196{,}624 / 1{,}216 \approx 161.7$.

The two convolution properties are: (1) **Parameter sharing**: same filter weights applied across all spatial positions; (2) **Local connectivity**: each output neuron connects to only $k \times k \times C_{\text{in}}$ inputs, not all.

B. Receptive field and dilation

For three stacked 3×3 convolutions, stride 1, no dilation:

(a) Using $r_\ell = r_{\ell-1} + (k_\ell - 1) \prod_{i<\ell} s_i$:

$$\begin{aligned} r_1 &= 1 + (3 - 1) \cdot 1 = 3 \\ r_2 &= 3 + (3 - 1) \cdot 1 = 5 \\ r_3 &= 5 + (3 - 1) \cdot 1 = 7 \end{aligned}$$

Receptive field after third layer: 7×7.

(b) With second layer dilation $d = 2$: Effective kernel size: $k_{\text{eff}} = k + (k - 1)(d - 1) = 3 + 2(1) = 5$.

$$\begin{aligned} r_1 &= 3 \\ r_2 &= 3 + (5 - 1) \cdot 1 = 7 \\ r_3 &= 7 + (3 - 1) \cdot 1 = 9 \end{aligned}$$

Receptive field: 9×9.

(c) Parameter count (per channel-pair): - Three 3×3 layers: $3 \times 9 = 27$ - One 7×7 layer: 49

Two reasons to prefer the stack: (1) fewer parameters (27 vs 49), and (2) more non-linearity (three ReLU activations vs one).

Table I
CUMULATIVE PRECISION/RECALL

#	TP/FP	Cum. precision	Cum. recall
1	TP	1/1 = 1.000	1/4 = 0.250
2	FP	1/2 = 0.500	1/4 = 0.250
3	TP	2/3 = 0.667	2/4 = 0.500
4	TP	3/4 = 0.750	3/4 = 0.750
5	FP	3/5 = 0.600	3/4 = 0.750

C. Transposed convolution for a decoder

A decoder upsamples $8 \times 8 \to 16 \times 16$ using:

$$n_{\text{out}} = s(n_{\text{in}} - 1) + k - 2P \tag{4}$$

(a) Choose $s = 2$: $16 = 2(8-1) + k - 2P = 14 + k - 2P$. Thus $k - 2P = 2$. Choose $k = 4, P = 1$: $4 - 2(1) = 2$. Valid $(k, s, P) = (4, 2, 1)$.

(b) Condition to avoid checkerboard artifacts: k must be divisible by s ($k\%s = 0$). My choice $k = 4, s = 2$ satisfies this. An alternative to transposed convolution is nearest-neighbor upsampling followed by a regular convolution.

D. Detection metrics: IoU, precision, recall

For boxes $A = (1,1,5,4)$ and $B = (3,2,7,6)$:

(a) Intersection: $x_1 = \max(1,3) = 3$, $y_1 = \max(1,2) = 2$, $x_2 = \min(5,7) = 5$, $y_2 = \min(4,6) = 4$. Area: $(5-3)(4-2) = 2 \times 2 = 4$.

Union: $12 + 16 - 4 = 24$.

IoU $= 4/24 = 1/6 \approx 0.1667$.

(b) Precision/Recall Table:

(c) Average Precision is the area under the precision-recall curve, computed via interpolation at each recall level. AP is preferred over a single precision/recall pair because it summarizes performance across all confidence thresholds, rather than depending on an arbitrary threshold choice.

E. Residual skip connections

(a) The degradation problem: As network depth increases, training accuracy saturates and then degrades rapidly — this is NOT overfitting, as deeper networks have higher training error, indicating the optimization problem is harder.

(b) For $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial F}{\partial \mathbf{x}} + \mathbf{I} \tag{5}$$

Even when $\partial F / \partial \mathbf{x}$ is tiny (near zero), the $+\mathbf{I}$ term ensures gradients flow through the skip connection, allowing early layers to receive signal in very deep networks.

(c) Driving $F \to 0$ (learning no change) is easier than learning an identity map through plain layers because plain layers have no built-in mechanism to bypass transformations.

Table II
SMALLCNN ARCHITECTURE

Stage	Layer	Output shape
Input	–	$3 \times 32 \times 32$
Block 1	Conv(3→32)–BN–ReLU–Pool(2)	$32 \times 16 \times 16$
Block 2	Conv(32→64)–BN–ReLU–Pool(2)	$64 \times 8 \times 8$
Block 3	Conv(64→128)–BN–ReLU–Pool(2)	$128 \times 4 \times 4$
Head	GAP + Linear(128→5)	5

F. Choosing a transfer-learning strategy

(i) 300 labelled images, similar to ImageNet: **Feature extraction** — freeze the backbone, train only a new head (limited data, but features are relevant).

(ii) 200k labelled images, similar classes: **Fine-tuning** — with abundant data, fine-tune the whole network to adapt domain-specific patterns without overfitting.

(iii) 500 grayscale medical scans, very different: **Partial fine-tuning** — freeze early layers (general features), fine-tune later layers or use feature extraction with a deeper head.

(a) The backbone's pretrained weights were optimized for specific input statistics (mean, std, and spatial size). Changing the input distribution changes what the weights see, effectively breaking the pretraining.

(b) Pretrained weights are already good. A large learning rate would make large updates that destroy these useful features. A smaller LR allows gentle adaptation without catastrophic forgetting.

III. EXPERIMENTAL SETUP

A. Data

I used a CIFAR-10 subset with 5 classes (airplane, automobile, bird, cat, deer), with 800 train and 200 test images per class, seeded by Student ID 121314. A validation set of 160 images per class was also extracted from the training data. For ResNet, images were resized to 224×224 and normalized with ImageNet mean/std (mean $[0.485, 0.456, 0.406]$, std $[0.229, 0.224, 0.225]$) as required by the pretrained backbone (Problem 6a).

B. Models

`SmallCNN` consists of 3 conv blocks (Conv–BatchNorm–ReLU–MaxPool), global average pooling, and a linear head. The architecture is summarized in Table II. For transfer learning, `build_resnet18` provides two modes: feature extraction (backbone frozen, new head only, 2,565 trainable params) and fine-tuning (whole network trainable, 11.2M params, smaller LR).

C. Augmentation

Mixup forms convex combinations:

$$\tilde{\mathbf{x}} = \lambda\, \mathbf{x}_i + (1-\lambda)\, \mathbf{x}_j, \tag{6}$$

$$\mathscr{L} = \lambda\, \ell(\hat{\mathbf{y}}, y_i) + (1-\lambda)\, \ell(\hat{\mathbf{y}}, y_j), \tag{7}$$

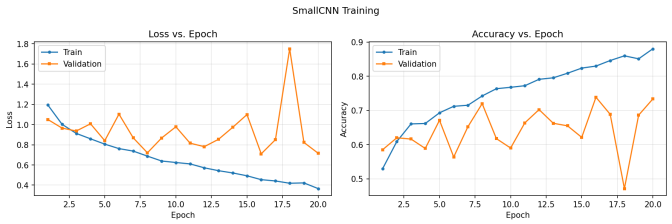

Fig. 1. Train/validation loss and accuracy vs. epoch for SmallCNN.

Table III
TRANSFER-LEARNING COMPARISON

Mode	Trainable params	Test acc.
Feature extraction (frozen)	2,565	0.8640
Fine-tuning (whole net)	11,179,077	0.9370

with $\lambda \sim \text{Beta}(1,1)$. I implemented Mixup (CutMix was also implemented but Mixup was used for the augmentation study as specified in B3).

D. Training and reproducibility

All experiments used Adam optimizer with cross-entropy loss, batch size 64, and 20 epochs. Learning rates were 10^{-3} for SmallCNN and feature extraction, and 10^{-4} for fine-tuning (Problem 6b). All randomness was fixed with Student ID 121314 as the seed, controlling data split, weight initialization, and shuffling, ensuring every figure regenerates via `run_all.py`.

IV. RESULTS

A. SmallCNN baseline (B1)

SmallCNN achieved a test accuracy of 70.30%. As shown in Fig. 1, the model converges steadily, but the gap between training and validation accuracy (88.06% vs 73.38%) indicates overfitting — the model memorizes training data but fails to generalize perfectly to unseen data.

B. Transfer learning: feature extraction vs. fine-tuning (B2)

Table III shows that fine-tuning significantly outperforms feature extraction (93.70% vs 86.40%) on this small dataset. The trainable parameter counts differ dramatically (2,565 vs 11.2M), but the additional capacity of fine-tuning is beneficial. As shown in Fig. 2, fine-tuning converges faster and achieves higher validation accuracy. This matches the transfer-learning guidance: with a relatively small but representative dataset, fine-tuning adapts the pretrained features to the specific domain more effectively than freezing them.

C. Data-augmentation study (B3)

Using the best architecture (fine-tuned ResNet-18), I trained under three regimes. Table IV and Fig. 3 show the results.

Surprisingly, no augmentation achieved the highest test accuracy (94.00%). Standard augmentation (RandomCrop + HorizontalFlip) performed worst (91.70%), while Mixup (93.40%)

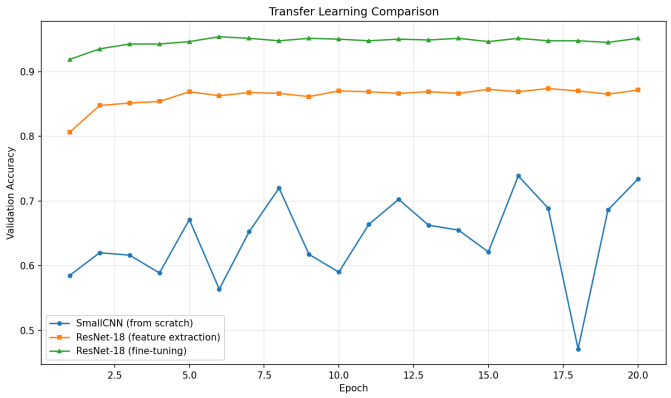

Fig. 2. Validation accuracy: feature extraction, fine-tuning, and the SmallCNN baseline, on one axis.

Table IV
AUGMENTATION STUDY

Regime	Test acc.
(a) None	0.9400
(b) Standard (crop + flip)	0.9170
(c) Mixup	0.9340

fell between the two. This suggests that with only 3,200 training images and a strong pretrained ResNet-18 backbone, augmentation may introduce unnecessary distribution shift rather than improving generalization. The validation curves confirm that "No Augmentation" consistently outperforms or matches the other regimes across all epochs, indicating a stable effect.

D. Summary and discussion (B4)

Transfer learning dramatically outperforms training from scratch (93.70% vs 70.30%) on this small dataset. Among transfer strategies, fine-tuning outperforms feature extraction. The augmentation study reveals that on this dataset, the pretrained features are already robust enough that no augmentation performed best — standard augmentation actually hurt performance, while Mixup provided a middle ground. The best overall setting was fine-tuned ResNet-18 with no augmentation (94.00%).

V. IMPLEMENTATION DISCUSSION

A. SmallCNN and global average pooling (C1)

My SmallCNN layers are:

- Block 1: Conv2d(3, 32, k=3, p=1) → BatchNorm → ReLU → MaxPool(2)
- Block 2: Conv2d(32, 64, k=3, p=1) → BatchNorm → ReLU → MaxPool(2)
- Block 3: Conv2d(64, 128, k=3, p=1) → BatchNorm → ReLU → MaxPool(2)
- Head: AdaptiveAvgPool2d(1) → Flatten → Linear(128, 5)

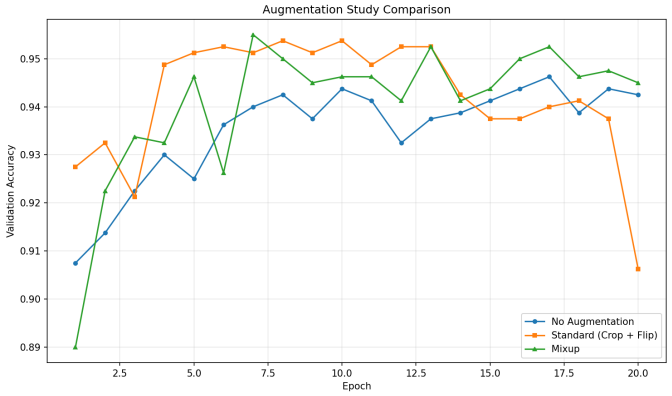

Fig. 3. Validation accuracy across the three augmentation regimes.

Table V
OVERALL RESULTS

Model	Setting	Test acc.
SmallCNN	from scratch	0.7030
ResNet-18	feature extraction	0.8640
ResNet-18	fine-tuning	0.9370
ResNet-18	No Augmentation	0.9400

Using (1): - Block 1: $32 \to (32-2)/2+1 = 16$ - Block 2: $16 \to (16-2)/2+1 = 8$ - Block 3: $8 \to (8-2)/2+1 = 4$

Spatial size at GAP input: 4×4.

I used GAP instead of flatten+dense because it reduces parameters from $128 \times 4 \times 4 \times 5 = 10,240$ to $128 \times 5 = 640$ (93.7% reduction), significantly reducing overfitting on the small dataset.

B. Freezing, concretely (C2)

In feature-extraction mode, I set:

```
for param in model.parameters():
    param.requires_grad = False
```

This freezes all ResNet-18 backbone parameters. The new 'model.fc' layer is created after this loop, so it has 'requires$_g$rad = True'bydefault.

Parameter counts (from 'count$_t$rainable$_p$arams()') : $-Featureextraction: 2,565(onlythenewLinear(512,5)head) - Fine-tuning: 11,179,077(entirenetwork)$

C. Mixup/CutMix in code (C3)

Mixup implementation from 'augment.py':

```
lam = np.random.beta(alpha, alpha)
idx = torch.randperm(batch_size)
x_mixed = lam * x + (1 - lam) * x[idx]
```

Loss combination:

```
def mix_criterion(loss_fn, logits, y_a, y_b, lam):
    return lam * loss_fn(logits, y_a) + (1 - lam) * loss_fn
        (logits, y_b)
```

The loss is a convex combination because $\lambda + (1-\lambda) = 1$. A single cross-entropy would only work for one label; the convex combination properly handles the mixed supervision since the input is a mix of two images.

D. A pitfall you hit (C4)

The most significant bug was in B3's test evaluation. Originally, I used 'load$_b$ase$_m$odel()'toloadtheB2checkpointforallthreeregimes,resultingini

E. Reproducibility (C5)

My Student ID is 121314. The seed flows through: 1. **Data split:** 'np.random.RandomState(seed)' controls shuffling and stratified sampling. 2. **Weight initialization:** PyTorch's default initialization is deterministic when 'torch.manual$_s$eed(seed)'isset.3.**DataLoadershuffling:** UsesPyTorch'sRNG,seededvia'torch.manual$_s$eed(seed)'.4. **CUDA:**'torch.cuda.manual$_s$eed$_a$ll(seed)'ensuresGPUdeterminism.

Running 'python run$_a$ll.py'twiceproducesidenticalfigures.

VI. PRETRAINED-MODEL INFERENCE (BONUS)

I ran pretrained torchvision models on 3 photos of soccer players (Ngolo Kante, Thomas Muller, and a match scene). The annotated outputs are saved as 'figures/zoo$_*$.png'.

Detection (Faster R-CNN with FPN): FPN (Feature Pyramid Network) uses a top-down pathway with lateral connections to build high-resolution semantic features at all scales, enabling detection of both small and large objects. NMS removes duplicate detections by keeping only the highest-confidence box per object.

Segmentation (Mask R-CNN with RoIAlign): RoIPool uses quantization (floor/ceil) causing misalignment between the RoI and feature map. RoIAlign uses bilinear interpolation to maintain sub-pixel precision, critical for accurate mask prediction.

Pose (Keypoint R-CNN with heatmap regression): Direct (x,y) regression is unstable because the loss landscape is non-convex with many local minima. Heatmap regression produces a 2D probability distribution that provides spatial context and is more robust to occlusions.

VII. CONCLUSION

The best performance on this CIFAR-10 subset was achieved by fine-tuning ResNet-18 without augmentation (94.00% test accuracy). Transfer learning proved dramatically superior to training from scratch (93.70% vs 70.30%). The augmentation study showed that on this small dataset with a strong pretrained backbone, standard augmentation can hurt performance, while Mixup provides a middle ground. The key takeaway is that transfer learning with carefully chosen hyperparameters is essential for small datasets, and augmentation strategies should be validated rather than assumed beneficial.

ACKNOWLEDGMENT: USE OF AI TOOLS

Used Claude (Anthropic) for: - Code structure and architecture planning - Debugging the B3 model checkpoint bug - Explain-

ing transposed convolution sizing - Generating the report template

All code was reviewed and understood before submission.

REFERENCES

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *NeurIPS*, 2012.

[3] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *ICLR*, 2015.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *CVPR*, 2016, pp. 770–778.

[5] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *ICML*, 2015.

[6] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *ICLR*, 2018.

[7] S. Yun *et al.*, "CutMix: Regularization strategy to train strong classifiers with localizable features," in *ICCV*, 2019.

[8] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *NeurIPS*, 2015.

[9] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *ICCV*, 2017.

[10] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *NeurIPS*, 2019.